KIESQUARE ANALYTICS

Data Engineering Internship Recruitment
Technical Assignment — 2026

Assignment Title	E-Commerce Behavior Data Engineering Pipeline
Dataset	eCommerce Behavior Data — Multi-Category Store
Submission Deadline	*14-04-2026*
Maximum Marks	100 Marks
Mode of Submission	GitHub Repository + Written Report (PDF)

1. Assignment Overview

This assignment evaluates your ability to design and implement a real-world data engineering pipeline using a large-scale e-commerce event dataset. You will demonstrate skills in data ingestion, transformation, SQL analytics, pipeline design, and data quality — all core competencies for a Data Engineering role.

Dataset: eCommerce Behavior Data
Source: https://drive.google.com/drive/folders/1MRcQPQScc-cq35Iw2ZHpMNzW--_hFOHx?usp=drive_link
Files: 2019-Oct.csv and 2019-Nov.csv
Events covered: view, cart, purchase across multiple product categories

2. Assignment Tasks

The assignment is divided into four sections. Sections A and B carry the highest weightage and must be completed with care. Each section builds upon the previous one.

Section A: Database Schema Design [35 Marks]

This is the most critical section of the assignment. You are expected to independently design a production-grade relational schema to store and query the e-commerce event data efficiently.

A1. Schema Design [20 Marks]
Explore the dataset thoroughly and design a normalised relational database schema to support the analytical queries in Section C. Your schema must:

1. Define all tables with column names, data types, primary keys, and foreign keys.
2. Be appropriately normalised — justify your choice of normal form (2NF, 3NF, or BCNF).
3. Include at least one fact table and at least two dimension tables.
4. Handle nullable fields correctly — document which columns allow NULLs and why.
5. Include indexes on columns you expect to be used frequently in WHERE, JOIN, and GROUP BY clauses. Justify each index.
6. Provide a clear Entity-Relationship (ER) diagram or schema diagram (drawn or generated).

What We Are Looking For
Schema design is not just about storing data — it is about making queries fast and the model easy to reason about.
We will assess whether your design would support the queries in Section C without full table scans.
A poorly normalised schema with no indexing strategy will score significantly lower regardless of correctness elsewhere.

A2. Schema Implementation [10 Marks]
Implement your schema in a database of your choice (PostgreSQL, MySQL, SQLite, DuckDB, or similar). Demonstrate:

7. DDL scripts (CREATE TABLE statements) for all tables with constraints.

8. Successful loading of a representative sample of the dataset into the schema (at least 5 million rows).
9. At least 3 queries that validate referential integrity and schema correctness.

A3. Schema Justification [5 Marks]

Write a short justification (200-300 words) explaining:

- Why you chose your specific schema structure over alternatives.
- What trade-offs you made (e.g., denormalisation for query speed vs. storage cost).
- How your index choices directly support the queries you plan to run.

Section B: ETL Pipeline & Performance Benchmarking [40 Marks]

Design, implement, and benchmark a complete ETL pipeline. Performance is a first-class requirement — you must measure and report it.

B1. ETL Pipeline Implementation [18 Marks]

Build a complete ETL pipeline that ingests both monthly CSV files, transforms the data, and loads it into your database schema from Section A. Your pipeline must:

10. Extract data from the raw CSV files with appropriate error handling for malformed rows.
11. Transform: deduplicate events, parse and standardise timestamps, handle NULLs per your schema rules, and derive any additional columns your schema requires.
12. Load into your database in batches — do not load everything in a single transaction.
13. Support incremental loading: running the pipeline again on the same file must not create duplicates (idempotency). Show proof with a re-run test.
14. Log the following for every pipeline run: start time, end time, rows extracted, rows transformed (dropped/modified), rows loaded, and any errors encountered.

B2. Performance Benchmarking [15 Marks]

This is a core deliverable. Measure and report the following performance metrics for your pipeline and database:

Metric	What to Measure & Report
Data Load Time	Total time to load October + November files end-to-end. Report separately per file and combined.
Batch Insert Throughput	Rows inserted per second for different batch sizes (e.g., 10k, 50k, 100k, 500k rows). Report optimal batch size.
Query Execution Time	Execution time for each analytical query in Section C — with and without indexes. Show the difference.
Memory Usage	Peak memory (RAM) consumed by your pipeline during the full load. Use Python's memory_profiler or equivalent.
Disk I/O	Size of raw input vs. size of data stored in your database / output format. Report compression ratio if applicable.
Index Impact	Compare query execution time on 3 queries before and after creating indexes. Tabulate the speedup factor.

Present all benchmarks in a dedicated Performance Report section in your README or notebook, with tables and charts.

B3. Data Quality Checks [7 Marks]

Implement a validation layer within your pipeline that checks for the following and logs violations:

- NULL values in columns defined as NOT NULL in your schema.
- Price values that are zero, negative, or unreasonably large (define your threshold and justify it).
- Event timestamps outside the expected data range.
- Duplicate events (same user, product, and timestamp combination).

Produce a Data Quality Summary: total rows processed, rows dropped, rows flagged, and overall pass rate (%).

Section C: Analytical Queries [15 Marks]

Write SQL queries against your loaded database to answer the following. For each query, report the execution time (from your benchmarks in B2).

15. Funnel analysis: For each product category, compute the count of distinct users at each stage — view, cart, purchase — and calculate the view-to-cart and cart-to-purchase conversion rates.
16. Session aggregation: For each user session, compute total events, distinct products interacted with, total cart value, and whether a purchase occurred.
17. Top 10 brands by total revenue (sum of purchase prices) for each month.
18. Identify users who purchased in October but did not return in November.
19. Hourly distribution of purchase events: which hour of the day has the highest purchase volume across the full dataset?

For each query, document: the SQL used, execution time with indexes, execution time without indexes, and the result set (first 10 rows).

Section D: Findings & Presentation [10 Marks]

D1. Visualisations [5 Marks]
Create at least 3 visualisations using any charting library. Suggested charts:

1. Batch size vs. insert throughput (rows/sec) — line chart from your B2 benchmarks.
2. Query execution time: with index vs. without index — bar chart for all 5 queries.
3. Conversion funnel (view → cart → purchase) for top 10 categories.

D2. Written Summary [5 Marks]
Write a concise summary (200-400 words) covering:

20. The key design decisions you made in your schema and why.
21. The most significant performance bottleneck you encountered and how you addressed it.
22. What you would do differently if the dataset were 10x larger (1TB+).

3. Evaluation Criteria

Your submission will be evaluated on the following parameters:

Criterion	What We Look For	Marks
DB Schema Design	Normalisation, ER diagram, index strategy, constraint correctness, justification	35
ETL Pipeline	Correctness, modularity, incremental load, idempotency, error handling	20
Performance Benchmarking	Load time, batch throughput, query speed, index impact, memory — all measured and reported	20
Data Quality	Comprehensive checks, logging, pass rate reporting	10
Analytical Queries	Correctness, execution time reported, result sets shown	10
Written Summary & Visuals	Clarity, design rationale, performance charts, scalability thinking	5
Bonus: Cloud / Orchestration	Pipeline deployed on cloud OR orchestrated with Airflow/Prefect	+5
TOTAL		100

4. Allowed Tools & Technologies

Category	Options
Language	Python 3.8+ (preferred) or SQL

Category	Options
Data Processing	Pandas, PySpark, DuckDB, Polars, SQLite
Notebooks	Jupyter Notebook, Google Colab, Databricks Community
Visualisation	Matplotlib, Seaborn, Plotly, Bokeh
Storage Format	Parquet or database tables; choose what fits your schema design
Cloud (optional)	AWS S3 + Glue, GCP BigQuery, Azure Data Factory
Orchestration (optional)	Apache Airflow, Prefect, or simple CLI scripts
Version Control	Git + GitHub (mandatory for submission)

5. Submission Guidelines

5.1 GitHub Repository Structure

Your repository must follow this folder structure:

```
your-name-data-engineering-assignment/
├── README.md                    # Setup instructions, benchmarks & summary
├── data/
│   └── raw/                     # Empty — add .gitignore; do not upload CSVs
├── schema/
│   ├── er_diagram.png           # Your ER / schema diagram
│   └── ddl.sql                  # CREATE TABLE statements
├── pipeline/
│   ├── extract.py
│   ├── transform.py
│   └── load.py
├── notebooks/
│   ├── 01_schema_design.ipynb
│   ├── 02_etl_pipeline.ipynb
│   ├── 03_benchmarks.ipynb
│   └── 04_queries.ipynb
├── reports/
│   └── performance_report.pdf
└── requirements.txt
```

5.2 README Requirements

Your README.md must include:

- Your full name, roll number, and branch.
- Step-by-step instructions to reproduce the pipeline from scratch.
- A performance benchmarks section with tables: load times, batch throughput, query times with/without indexes.
- Schema design decisions and any assumptions or known limitations.

5.3 What NOT to Submit

- Do NOT upload the raw CSV files to GitHub (file sizes exceed GitHub limits).
- Do NOT hard-code absolute file paths — use relative paths or config variables.
- Do NOT submit notebooks with unexecuted cells — run all cells before submitting.

Academic Integrity Notice
This is an individual assignment. Collaboration on code or analysis is not permitted.
You may reference public notebooks and documentation but all submitted code must be your own.
Plagiarism detection tools will be used. Any similarity above 30% with external sources will result in disqualification.
Use of AI coding assistants is permitted for syntax help, but the design and logic must be your own.

We wish you the best of luck!

This assignment is designed to reflect real-world data engineering challenges. Focus on correctness, clean design, and clear communication of your findings.